

InternNova Internship

Week 2 Assignment Report

Student Name: Hrithik Thakur. A

Domain: Data Analytics

Batch: Batch-03

Table of Contents

Task 1: NumPy Introduction & Arrays — 10 Marks

Task 2: NumPy Indexing, Slicing & Reshaping — 10 Marks

Task 3: NumPy Mathematical & Statistical Operations —10 Marks

Task 4: Pandas Series & Data_Frame — 10 Marks

Task 5: Reading & Inspecting Data — 10 Marks

Task 6: Selecting, Filtering & Sorting Data —10 Marks

Task 7: Handling Missing Values — 10 Marks

Task 8: Merge, Concatenate, GroupBy & Pivot Table — 15 Marks

Task 9: Exporting Data — 5 Marks

Task 10: Mini Data Analysis Project (incl. Visualizations) — 10 Marks

Datasets Used

Two CSV datasets were created for the Assignment: student.csv (20 student records with intentional missing values in the marks/attendance columns, used for tasks 5-7, 9, and 10) And departments.csv (branch-wise department info, used for the merge operation in Task 8).

students.csv

Student_id	Name	Branch	Marks	Attendance	City	
101	Aarav Sharma	CSE	88	92	Delhi	
102	Priya Verma	ECE	76	85	Mumbai	
103	Rohan Gupta	CSE		78	Delhi	
104	Sneha Iyer	ME	65	90	Chennai	
105	Karan Mehta	ECE	91		Mumbai	
106	Divya Nair	CSE	58	70	Bangalore	
107	Arjun Reddy	ME	73	88	Hyderabad	
108	Ananya Singh	ECE	84	95	Delhi	
109	Vikram Rao	CSE		60	Bangalore	
110	Ishita Kapoor	ME	95	98	Chennai	
111	Rahul Joshi	CSE	69	82	Mumbai	
112	Meera Pillai	ECE		91	Chennai	
113	Aditya Kumar	ME	80	87	Delhi	
114	Neha Bhatt	CSE	72	79	Bangalore	
115	Siddharth Rao	ECE	89	93	Hyderabad	
116	Anjali Desai	ME		85	Mumbai	
117	Varun Malhotra	CSE	77	88	Delhi	
118	Shreya Nair	ECE	82		Bangalore	
119	Rahul Sharma	ME	68	75	Chennai	
120	Simran Kaur	CSE	90		Mumbai	

Departments.csv

Branch	HOD	Building
CSE	Dr. Anil Kapoor	Block A
ECE	Dr. Sunita Rao	Block B
ME	Dr. Ramesh Iyer	Block C

Assignment Tasks, Source Code & Outputs

Task 1: NumPy Introduction & Arrays

(10 Marks)

Explanation: This program imports NumPy, creates a 1D array of 11 numbers, and displays its shape, size, and data type. It also demonstrates the creation of a simple one-dimensional array and a two-dimensional (3x3) array.

Source Code — task1.py

```
import numpy as np

# 1D array with at least 10 numbers

arr = np.array([12, 45, 7, 23, 56, 89, 34, 21, 67, 90, 15])

print("----- 1D NumPy Array -----")

print("Array :", arr)

print("Shape :", arr.shape)

print("Size :", arr.size)

print("Data type :", arr.dtype)

# One-dimensional array

one_d = np.array([1, 2, 3, 4, 5])

print("\n----- One-Dimensional Array -----")

print(one_d)

print("Shape:", one_d.shape)

# Two-dimensional array

two_d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print("\n----- Two-Dimensional Array -----")

print(two_d)

print("Shape:", two_d.shape)
```

Output

```
----- 1D NumPy Array -----
Array : [12 45  7 23 56 89 34 21 67 90 15]
Shape  : (11,)
Size   : 11
Data type : int64

----- One-Dimensional Array -----
[1 2 3 4 5]
Shape: (5,)

----- Two-Dimensional Array -----
[[1 2 3]
 [4 5 6]
 [7 8 9]]
Shape: (3, 3)
hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %
```

Task 2: NumPy Indexing, Slicing & Reshaping

(10 Marks)

Explanation: This program demonstrates indexing (accessing specific elements), slicing (extracting sub-arrays), converting a 1D array into a 2D array, accessing specific rows/columns, and reshaping an array into multiple different

Source Code — task2.py

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120])

print("----- Original Array -----")
print(arr)

# Indexing
print("\n----- Indexing -----")
print("Element at index 0 :", arr[0])
print("Element at index 5 :", arr[5])
print("Last element :", arr[-1])

# Slicing
print("\n----- Slicing -----")
print("First 4 elements :", arr[:4])
print("Elements 3 to 7 :", arr[3:8])
print("Every 2nd element :", arr[::2])

# 2D array
arr_2d = arr.reshape(3, 4)
print("\n----- 2D Array (reshaped to 3x4) -----")
print(arr_2d)

# Accessing rows and columns
print("\n----- Row/Column Access -----")
print("Row 0 :", arr_2d[0])
```

```

print("Row 1 :", arr_2d[1])

print("Column 2 :", arr_2d[:, 2])

print("Element at (2, 3) :", arr_2d[2, 3])


# Reshaping into different dimensions

reshaped_2x6 = arr.reshape(2, 6)

reshaped_4x3 = arr.reshape(4, 3)

print("\n----- Reshaping -----")

print("Original shape :", arr.shape)

print("Reshaped to (2,6):\n", reshaped_2x6)

print("Reshaped to (4,3):\n", reshaped_4x3)

```

Output

```

----- Original Array -----
[ 10  20  30  40  50  60  70  80  90 100 110 120]

----- Indexing -----
Element at index 0 : 10
Element at index 5 : 60
Last element      : 120

----- Slicing -----
First 4 elements   : [10 20 30 40]
Elements 3 to 7    : [40 50 60 70 80]
Every 2nd element  : [ 10  30  50  70  90 110]

----- 2D Array (reshaped to 3x4) -----
[[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]

----- Row/Column Access -----
Row 0              : [10 20 30 40]
Row 1              : [50 60 70 80]
Column 2           : [ 30  70 110]
Element at (2, 3)  : 120

----- Reshaping -----
Original shape      : (12,)
Reshaped to (2,6):
[[ 10  20  30  40  50  60]
 [ 70  80  90 100 110 120]]
Reshaped to (4,3):
[[ 10  20  30]
 [ 40  50  60]
 [ 70  80  90]
 [100 110 120]]
hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %

```

Task 3: NumPy Mathematical & Statistical Operations

(10 Marks)

Explanation: Using two numeric arrays representing exam and bonus marks, this program performs element-wise addition, subtraction, multiplication, and division, and calculates statistical measures (mean, median, min, max, standard deviation, sum) on the exam marks.

Source Code — task3.py

```
import numpy as np

marks_a = np.array([88, 76, 65, 91, 58, 73, 84, 95, 69, 80])

marks_b = np.array([5, 4, 3, 6, 2, 4, 5, 3, 4, 2]) # bonus marks

print("----- Dataset -----")

print("Marks A (exam) :", marks_a)

print("Marks B (bonus):", marks_b)

# Mathematical operations

print("\n----- Mathematical Operations -----")

print("Addition :", marks_a + marks_b)

print("Subtraction :", marks_a - marks_b)

print("Multiplication :", marks_a * marks_b)

print("Division :", np.round(marks_a / marks_b, 2))

# Statistical functions

print("\n----- Statistical Functions (Marks A) -----")

print("Mean :", np.mean(marks_a))

print("Median :", np.median(marks_a))

print("Minimum :", np.min(marks_a))

print("Maximum :", np.max(marks_a))

print("Standard Dev. :", round(np.std(marks_a), 2))

print("Sum :", np.sum(marks_a))
```

Output

```
----- Dataset -----
Marks A (exam) : [88 76 65 91 58 73 84 95 69 80]
Marks B (bonus): [5 4 3 6 2 4 5 3 4 2]

----- Mathematical Operations -----
Addition      : [93 80 68 97 60 77 89 98 73 82]
Subtraction   : [83 72 62 85 56 69 79 92 65 78]
Multiplication : [440 304 195 546 116 292 420 285 276 160]
Division      : [17.6  19.   21.67 15.17 29.   18.25 16.8  31.67 17.25 40.   ]

----- Statistical Functions (Marks A) -----
Mean          : 77.9
Median        : 78.0
Minimum       : 58
Maximum       : 95
Standard Dev. : 11.3
Sum           : 779
hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %
```

Task 4: Pandas Series & DataFrame

(10 Marks)

Explanation: This program creates a Pandas Series of marks, builds a DataFrame containing student information (ID, name, branch, marks), displays its column names and index, and adds a new 'grade' column derived from the marks.

Source Code — task4.py

```
import pandas as pd

# Pandas Series

marks_series = pd.Series([89, 76, 65, 91, 58], name="marks")

print("----- Pandas Series -----")

print(marks_series)

# DataFrame with student information

data = {
    "student_id": [101, 102, 103, 104, 105],
    "name": ["Aarav Sharma", "Priya Verma", "Rohan Gupta", "Sneha Iyer", "Karan Mehta"],
    "branch": ["CSE", "ECE", "CSE", "ME", "ECE"],
    "marks": [88, 76, 65, 91, 58]
}

df = pd.DataFrame(data)

print("\n----- Student DataFrame -----")

print(df)

print("\n----- Column Names -----")

print(df.columns.tolist())

print("\n----- Index -----")

print(df.index)

# Adding a new column

df["grade"] = df["marks"].apply(lambda m: "A" if m >= 85 else "B" if m >= 70 else "C")

print("\n----- Updated DataFrame (with 'grade' column) -----")

print(df)
```

Output

```
----- Pandas Series -----
0      88
1      76
2      65
3      91
4      58
Name: marks, dtype: int64

----- Student DataFrame -----
   student_id      name branch  marks
0         101  Aarav Sharma   CSE    88
1         102   Priya Verma   ECE    76
2         103   Rohan Gupta   CSE    65
3         104    Sneha Iyer    ME    91
4         105   Karan Mehta   ECE    58

----- Column Names -----
['student_id', 'name', 'branch', 'marks']

----- Index -----
RangeIndex(start=0, stop=5, step=1)

----- Updated DataFrame (with 'grade' column) -----
   student_id      name branch  marks grade
0         101  Aarav Sharma   CSE    88    A
1         102   Priya Verma   ECE    76    B
2         103   Rohan Gupta   CSE    65    C
3         104    Sneha Iyer    ME    91    A
4         105   Karan Mehta   ECE    58    C
hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %
```

Task 5: Reading & Inspecting Data

(10 Marks)

Explanation: This program reads the students.csv dataset using Pandas and inspects it using head(), tail(), shape, columns, dtypes, info(), and describe() to understand the structure and statistical summary of the data.

Source Code — task5.py

```
import pandas as pd

df = pd.read_csv("students.csv")

print("----- First 5 Rows (head) -----")
print(df.head())

print("\n----- Last 5 Rows (tail) -----")
print(df.tail())

print("\n----- Shape (rows, columns) -----")
print(df.shape)

print("\n----- Column Names -----")
print(df.columns.tolist())

print("\n----- Data Types -----")
print(df.dtypes)

print("\n----- info() -----")
df.info()

print("\n----- describe() -----")
print(df.describe())
```

Output:

```
----- First 5 Rows (head) -----
  Student_id  Name  Branch  Marks  Attendance  City
0         101  Aarav Sharma  CSE   88.0       92.0  Delhi
1         102  Priya Verma   ECE   76.0       85.0  Mumbai
2         103  Rohan Gupta   CSE   NaN       78.0  Delhi
3         104  Sneha Iyer    ME    65.0       90.0  Chennai
4         105  Karan Mehta   ECE   91.0       NaN  Mumbai

----- Last 5 Rows (tail) -----
  Student_id  Name  Branch  Marks  Attendance  City
15         116  Anjali Desai   ME   NaN       85.0  Mumbai
16         117  Varun Malhotra CSE   77.0       88.0  Delhi
17         118  Shreya Nair    ECE   82.0       NaN  Bangalore
18         119  Rahul Sharma   ME   68.0       75.0  Chennai
19         120  Simran Kaur    CSE   90.0       NaN  Mumbai

----- Shape (rows, columns) -----
(20, 6)

----- Column Names -----
['Student_id', 'Name', 'Branch', 'Marks', 'Attendance', 'City']

----- Data Types -----
Student_id    int64
Name          object
Branch        object
Marks         float64
Attendance    float64
City          object
dtype: object

----- info() -----
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Student_id  20 non-null    int64
1   Name        20 non-null    object
2   Branch      20 non-null    object
3   Marks       16 non-null    float64
4   Attendance  17 non-null    float64
5   City        20 non-null    object
dtypes: float64(2), int64(1), object(3)
memory usage: 1.1+ KB

----- describe() -----
         Student_id      Marks  Attendance
count    20.000000   16.000000    17.000000
mean     110.500000   78.562500    84.470588
std        5.91608    10.614259     9.696118
min      101.000000   58.000000    60.000000
25%      105.750000   71.250000    79.000000
50%      110.500000   78.500000    87.000000
75%      115.250000   88.250000    91.000000
max      120.000000   95.000000    98.000000
hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %
```

Task 6: Selecting, Filtering & Sorting Data

(10 Marks)

Explanation: This program demonstrates selecting specific columns and rows from the student dataset, filtering records using single and multiple conditions (e.g., marks > 80, CSE students with marks > 70), and sorting data in both ascending and descending order by marks.

Source Code — task6.py

```
import pandas as pd

df = pd.read_csv("students.csv")

# Select specific columns
print("----- Selecting Specific Columns (name, marks) -----")
print(df[["name", "marks"]])

# Select specific rows
print("\n----- Selecting Specific Rows (index 0 to 3) -----")
print(df.iloc[0:4])

# Filter based on a single condition
print("\n----- Students with marks > 80 -----")
print(df[df["marks"] > 80])

# Multiple filtering conditions
print("\n----- CSE Students with marks > 70 -----")
print(df[(df["branch"] == "CSE") & (df["marks"] > 70)])

# Sort ascending
print("\n----- Sorted by marks (Ascending) -----")
print(df.sort_values(by="marks", ascending=True))

# Sort descending
```

```
print("\n----- Sorted by marks (Descending) -----")

print(df.sort_values(by="marks", ascending=False))
```

Output

```
----- Selecting Specific Columns (name, marks) -----
      Name  Marks
0  Aarav Sharma  88.0
1   Priya Verma  76.0
2   Rohan Gupta   NaN
3   Sneha Iyer   65.0
4   Karan Mehta  91.0
5   Divya Nair   58.0
6   Arjun Reddy  73.0
7   Ananya Singh  84.0
8   Vikram Rao   NaN
9   Ishita Kapoor  95.0
10  Rahul Joshi  69.0
11  Meera Pillai   NaN
12  Aditya Kumar  80.0
13   Neha Bhatt  72.0
14  Siddharth Rao  89.0
15   Anjali Desai   NaN
16  Varun Malhotra  77.0
17   Shreya Nair  82.0
18  Rahul Sharma  68.0
19   Simran Kaur  90.0

----- Selecting Specific Rows (index 0 to 3) -----
      Student_id      Name Branch  Marks  Attendance  City
0           101  Aarav Sharma   CSE   88.0         92.0  Delhi
1           102   Priya Verma   ECE   76.0         85.0  Mumbai
2           103   Rohan Gupta   CSE    NaN         78.0  Delhi
3           104   Sneha Iyer    ME   65.0         90.0  Chennai

----- Students with marks > 80 -----
      Student_id      Name Branch  Marks  Attendance  City
0           101  Aarav Sharma   CSE   88.0         92.0  Delhi
4           105   Karan Mehta   ECE   91.0          NaN  Mumbai
7           108   Ananya Singh   ECE   84.0         95.0  Delhi
9           110  Ishita Kapoor    ME   95.0         98.0  Chennai
14          115  Siddharth Rao   ECE   89.0         93.0  Hyderabad
17          118   Shreya Nair   ECE   82.0          NaN  Bangalore
19          120   Simran Kaur   CSE   90.0          NaN  Mumbai

----- CSE Students with marks > 70 -----
      Student_id      Name Branch  Marks  Attendance  City
0           101  Aarav Sharma   CSE   88.0         92.0  Delhi
13          114   Neha Bhatt   CSE   72.0         79.0  Bangalore
16          117  Varun Malhotra   CSE   77.0         88.0  Delhi
19          120   Simran Kaur   CSE   90.0          NaN  Mumbai
```

----- Sorted by marks (Ascending) -----

	Student_id	Name	Branch	Marks	Attendance	City
5	106	Divya Nair	CSE	58.0	70.0	Bangalore
3	104	Sneha Iyer	ME	65.0	90.0	Chennai
18	119	Rahul Sharma	ME	68.0	75.0	Chennai
10	111	Rahul Joshi	CSE	69.0	82.0	Mumbai
13	114	Neha Bhatt	CSE	72.0	79.0	Bangalore
6	107	Arjun Reddy	ME	73.0	88.0	Hyderabad
1	102	Priya Verma	ECE	76.0	85.0	Mumbai
16	117	Varun Malhotra	CSE	77.0	88.0	Delhi
12	113	Aditya Kumar	ME	80.0	87.0	Delhi
17	118	Shreya Nair	ECE	82.0	NaN	Bangalore
7	108	Ananya Singh	ECE	84.0	95.0	Delhi
0	101	Aarav Sharma	CSE	88.0	92.0	Delhi
14	115	Siddharth Rao	ECE	89.0	93.0	Hyderabad
19	120	Simran Kaur	CSE	90.0	NaN	Mumbai
4	105	Karan Mehta	ECE	91.0	NaN	Mumbai
9	110	Ishita Kapoor	ME	95.0	98.0	Chennai
2	103	Rohan Gupta	CSE	NaN	78.0	Delhi
8	109	Vikram Rao	CSE	NaN	60.0	Bangalore
11	112	Meera Pillai	ECE	NaN	91.0	Chennai
15	116	Anjali Desai	ME	NaN	85.0	Mumbai

----- Sorted by marks (Descending) -----

	Student_id	Name	Branch	Marks	Attendance	City
9	110	Ishita Kapoor	ME	95.0	98.0	Chennai
4	105	Karan Mehta	ECE	91.0	NaN	Mumbai
19	120	Simran Kaur	CSE	90.0	NaN	Mumbai
14	115	Siddharth Rao	ECE	89.0	93.0	Hyderabad
0	101	Aarav Sharma	CSE	88.0	92.0	Delhi
7	108	Ananya Singh	ECE	84.0	95.0	Delhi
17	118	Shreya Nair	ECE	82.0	NaN	Bangalore
12	113	Aditya Kumar	ME	80.0	87.0	Delhi
16	117	Varun Malhotra	CSE	77.0	88.0	Delhi
1	102	Priya Verma	ECE	76.0	85.0	Mumbai
6	107	Arjun Reddy	ME	73.0	88.0	Hyderabad
13	114	Neha Bhatt	CSE	72.0	79.0	Bangalore
10	111	Rahul Joshi	CSE	69.0	82.0	Mumbai
18	119	Rahul Sharma	ME	68.0	75.0	Chennai
3	104	Sneha Iyer	ME	65.0	90.0	Chennai
5	106	Divya Nair	CSE	58.0	70.0	Bangalore
2	103	Rohan Gupta	CSE	NaN	78.0	Delhi
8	109	Vikram Rao	CSE	NaN	60.0	Bangalore
11	112	Meera Pillai	ECE	NaN	91.0	Chennai
15	116	Anjali Desai	ME	NaN	85.0	Mumbai

hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files % █

Task 7: Handling Missing Values

(10 Marks)

Explanation: This program identifies missing values using `isnull()`, counts missing values per column, removes rows with missing data using `dropna()`, and fills missing numeric values with the column mean using `fillna()`. It also explains why handling missing data matters in data analytics.

Source Code — task7.py

```
import pandas as pd

df = pd.read_csv("students.csv")

print("----- Dataset Before Handling Missing Values -----")
print(df)

# Identify missing values
print("\n----- Missing Values (True/False) -----")
print(df.isnull())

# Count missing values per column
print("\n----- Count of Missing Values per Column -----")
print(df.isnull().sum())

# Remove rows containing missing values
df_dropped = df.dropna()
print("\n----- Dataset After Dropping Rows with Missing Values -----")
print(df_dropped)

# Fill missing values (numeric columns filled with column mean)
df_filled = df.copy()
df_filled["Marks"] = df_filled["Marks"].fillna(round(df_filled["Marks"].mean(), 1))
df_filled["Attendance"] = df_filled["Attendance"].fillna(round(df_filled["Attendance"].mean(), 1))
```

```
print("\n----- Dataset After Filling Missing Values (with column mean) -----")
print(df_filled)
print("""----- Why Handling Missing Data Matters in Data Analytics -----
Missing values can distort statistical calculations (mean, sum, correlations),
break machine learning models that cannot handle NaN values, and lead to biased
or incorrect conclusions if ignored. Properly identifying and handling missing
data (via removal or imputation) ensures the dataset remains accurate, complete,
and reliable for further analysis and decision-making.""")
```

Output

```
----- Dataset Before Handling Missing Values -----
Student_id  Name  Branch  Marks  Attendance  City
0          101  Aarav Sharma  CSE    88.0         92.0   Delhi
1          102  Priya Verma   ECE    76.0         85.0   Mumbai
2          103  Rohan Gupta   CSE    NaN          78.0   Delhi
3          104  Sneha Iyer    ME     65.0         90.0   Chennai
4          105  Karan Mehta   ECE    91.0         NaN    Mumbai
5          106  Divya Nair    CSE    58.0         70.0   Bangalore
6          107  Arjun Reddy   ME     73.0         88.0   Hyderabad
7          108  Ananya Singh  ECE    84.0         95.0   Delhi
8          109  Vikram Rao    CSE    NaN          60.0   Bangalore
9          110  Ishita Kapoor ME     95.0         98.0   Chennai
10         111  Rahul Joshi   CSE    69.0         82.0   Mumbai
11         112  Meera Pillai ECE    NaN          91.0   Chennai
12         113  Aditya Kumar ME     80.0         87.0   Delhi
13         114  Neha Bhatt   CSE    72.0         79.0   Bangalore
14         115  Siddharth Rao ECE    89.0         93.0   Hyderabad
15         116  Anjali Desai ME     NaN          85.0   Mumbai
16         117  Varun Malhotra CSE    77.0         88.0   Delhi
17         118  Shreya Nair   ECE    82.0         NaN    Bangalore
18         119  Rahul Sharma ME     68.0         75.0   Chennai
19         120  Simran Kaur  CSE    90.0         NaN    Mumbai
```

```
----- Missing Values (True/False) -----
Student_id  Name  Branch  Marks  Attendance  City
0          False False False False False False
1          False False False False False False
2          False False False True False False
3          False False False False False False
4          False False False False True False
5          False False False False False False
6          False False False False False False
7          False False False False False False
8          False False False True False False
9          False False False False False False
10         False False False False False False
11         False False False True False False
12         False False False False False False
13         False False False False False False
14         False False False False False False
15         False False False True False False
16         False False False False False False
17         False False False False True False
18         False False False False False False
19         False False False False True False
```

----- Count of Missing Values per Column -----

```
Student_id    0
Name          0
Branch        0
Marks         4
Attendance    3
City          0
dtype: int64
```

----- Dataset After Dropping Rows with Missing Values -----

	Student_id	Name	Branch	Marks	Attendance	City
0	101	Aarav Sharma	CSE	88.0	92.0	Delhi
1	102	Priya Verma	ECE	76.0	85.0	Mumbai
3	104	Sneha Iyer	ME	65.0	90.0	Chennai
5	106	Divya Nair	CSE	58.0	70.0	Bangalore
6	107	Arjun Reddy	ME	73.0	88.0	Hyderabad
7	108	Ananya Singh	ECE	84.0	95.0	Delhi
9	110	Ishita Kapoor	ME	95.0	98.0	Chennai
10	111	Rahul Joshi	CSE	69.0	82.0	Mumbai
12	113	Aditya Kumar	ME	80.0	87.0	Delhi
13	114	Neha Bhatt	CSE	72.0	79.0	Bangalore
14	115	Siddharth Rao	ECE	89.0	93.0	Hyderabad
16	117	Varun Malhotra	CSE	77.0	88.0	Delhi
18	119	Rahul Sharma	ME	68.0	75.0	Chennai

----- Dataset After Filling Missing Values (with column mean) -----

	Student_id	Name	Branch	Marks	Attendance	City
0	101	Aarav Sharma	CSE	88.0	92.0	Delhi
1	102	Priya Verma	ECE	76.0	85.0	Mumbai
2	103	Rohan Gupta	CSE	78.6	78.0	Delhi
3	104	Sneha Iyer	ME	65.0	90.0	Chennai
4	105	Karan Mehta	ECE	91.0	84.5	Mumbai
5	106	Divya Nair	CSE	58.0	70.0	Bangalore
6	107	Arjun Reddy	ME	73.0	88.0	Hyderabad
7	108	Ananya Singh	ECE	84.0	95.0	Delhi
8	109	Vikram Rao	CSE	78.6	60.0	Bangalore
9	110	Ishita Kapoor	ME	95.0	98.0	Chennai
10	111	Rahul Joshi	CSE	69.0	82.0	Mumbai
11	112	Meera Pillai	ECE	78.6	91.0	Chennai
12	113	Aditya Kumar	ME	80.0	87.0	Delhi
13	114	Neha Bhatt	CSE	72.0	79.0	Bangalore
14	115	Siddharth Rao	ECE	89.0	93.0	Hyderabad
15	116	Anjali Desai	ME	78.6	85.0	Mumbai
16	117	Varun Malhotra	CSE	77.0	88.0	Delhi
17	118	Shreya Nair	ECE	82.0	84.5	Bangalore
18	119	Rahul Sharma	ME	68.0	75.0	Chennai
19	120	Simran Kaur	CSE	90.0	84.5	Mumbai

----- Why Handling Missing Data Matters in Data Analytics -----

Missing values can distort statistical calculations (mean, sum, correlations), break machine learning models that cannot handle NaN values, and lead to biased or incorrect conclusions if ignored. Properly identifying and handling missing data (via removal or imputation) ensures the dataset remains accurate, complete, and reliable for further analysis and decision-making.

hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %

Task 8: Merge, Concatenate, GroupBy & Pivot Table

(15 Marks)

Explanation: Using students.csv and departments.csv, this program merges the two datasets on the common 'branch' column, concatenates two halves of the student dataset, performs a GroupBy aggregation (mean/count/min/max marks by branch), and builds a Pivot Table summarizing average marks by branch and city.

Source Code — task8.py

```
import pandas as pd

students = pd.read_csv("students.csv")

departments = pd.read_csv("departments.csv")

print("----- Students Dataset -----")

print(students)

print("\n----- Departments Dataset -----")

print(departments)

# Merge on the common column 'branch'

merged = pd.merge(students, departments, on="Branch", how="left")

print("\n----- Merged DataFrame (students + departments on 'Branch') -----")

print(merged[["Student_id", "Name", "Branch", "HOD", "Building"]])

# Concatenate two DataFrames (splitting students into two halves as an example)

first_half = students.iloc[:7]

second_half = students.iloc[7:]

concatenated = pd.concat([first_half, second_half], ignore_index=True)

print("\n----- Concatenated DataFrame (two halves of students) -----")

print(concatenated)

# GroupBy: group by branch and calculate aggregate marks

print("\n----- GroupBy: Average, Count, Min, Max Marks per Branch -----")

grouped = students.groupby("Branch")["Marks"].agg(["mean", "count", "min", "max"])

print(grouped)

# Pivot Table: average marks and attendance per branch and city

print("\n----- Pivot Table: Average Marks by Branch and City -----")

pivot = pd.pivot_table(

students,
```

```

values="Marks",

index="Branch",

columns="City",

aggfunc="mean"

)

print(pivot)

```

Output

```

----- Students Dataset -----
Student_id  Name  Branch  Marks  Attendance  City
0          101  Aarav Sharma  CSE    88.0        92.0    Delhi
1          102  Priya Verma    ECE    76.0        85.0    Mumbai
2          103  Rohan Gupta    CSE     NaN        78.0    Delhi
3          104  Sneha Iyer      ME    65.0        90.0    Chennai
4          105  Karan Mehta    ECE    91.0         NaN    Mumbai
5          106  Divya Nair     CSE    58.0        70.0    Bangalore
6          107  Arjun Reddy    ME    73.0        88.0    Hyderabad
7          108  Ananya Singh   ECE    84.0        95.0    Delhi
8          109  Vikram Rao     CSE     NaN        60.0    Bangalore
9          110  Ishita Kapoor  ME    95.0        98.0    Chennai
10         111  Rahul Joshi    CSE    69.0        82.0    Mumbai
11         112  Meera Pillai   ECE     NaN        91.0    Chennai
12         113  Aditya Kumar   ME    80.0        87.0    Delhi
13         114  Neha Bhatt     CSE    72.0        79.0    Bangalore
14         115  Siddharth Rao  ECE    89.0        93.0    Hyderabad
15         116  Anjali Desai   ME     NaN        85.0    Mumbai
16         117  Varun Malhotra CSE    77.0        88.0    Delhi
17         118  Shreya Nair    ECE    82.0         NaN    Bangalore
18         119  Rahul Sharma   ME    68.0        75.0    Chennai
19         120  Simran Kaur    CSE    90.0         NaN    Mumbai

----- Departments Dataset -----
Branch  HOD  Building
0  CSE  Dr. Anil Kapoor  Block A
1  ECE  Dr. Sunita Rao  Block B
2  ME  Dr. Ramesh Iyer  Block C

----- Merged DataFrame (students + departments on 'Branch') -----
Student_id  Name  Branch  HOD  Building
0          101  Aarav Sharma  CSE  Dr. Anil Kapoor  Block A
1          102  Priya Verma    ECE  Dr. Sunita Rao  Block B
2          103  Rohan Gupta    CSE  Dr. Anil Kapoor  Block A
3          104  Sneha Iyer      ME  Dr. Ramesh Iyer  Block C
4          105  Karan Mehta    ECE  Dr. Sunita Rao  Block B
5          106  Divya Nair     CSE  Dr. Anil Kapoor  Block A
6          107  Arjun Reddy    ME  Dr. Ramesh Iyer  Block C
7          108  Ananya Singh   ECE  Dr. Sunita Rao  Block B
8          109  Vikram Rao     CSE  Dr. Anil Kapoor  Block A
9          110  Ishita Kapoor  ME  Dr. Ramesh Iyer  Block C
10         111  Rahul Joshi    CSE  Dr. Anil Kapoor  Block A
11         112  Meera Pillai   ECE  Dr. Sunita Rao  Block B
12         113  Aditya Kumar   ME  Dr. Ramesh Iyer  Block C
13         114  Neha Bhatt     CSE  Dr. Anil Kapoor  Block A
14         115  Siddharth Rao  ECE  Dr. Sunita Rao  Block B
15         116  Anjali Desai   ME  Dr. Ramesh Iyer  Block C
16         117  Varun Malhotra CSE  Dr. Anil Kapoor  Block A
17         118  Shreya Nair    ECE  Dr. Sunita Rao  Block B
18         119  Rahul Sharma   ME  Dr. Ramesh Iyer  Block C
19         120  Simran Kaur    CSE  Dr. Anil Kapoor  Block A

```

----- Concatenated DataFrame (two halves of students) -----

	Student_id	Name	Branch	Marks	Attendance	City
0	101	Aarav Sharma	CSE	88.0	92.0	Delhi
1	102	Priya Verma	ECE	76.0	85.0	Mumbai
2	103	Rohan Gupta	CSE	NaN	78.0	Delhi
3	104	Sneha Iyer	ME	65.0	90.0	Chennai
4	105	Karan Mehta	ECE	91.0	NaN	Mumbai
5	106	Divya Nair	CSE	58.0	70.0	Bangalore
6	107	Arjun Reddy	ME	73.0	88.0	Hyderabad
7	108	Ananya Singh	ECE	84.0	95.0	Delhi
8	109	Vikram Rao	CSE	NaN	60.0	Bangalore
9	110	Ishita Kapoor	ME	95.0	98.0	Chennai
10	111	Rahul Joshi	CSE	69.0	82.0	Mumbai
11	112	Meera Pillai	ECE	NaN	91.0	Chennai
12	113	Aditya Kumar	ME	80.0	87.0	Delhi
13	114	Neha Bhatt	CSE	72.0	79.0	Bangalore
14	115	Siddharth Rao	ECE	89.0	93.0	Hyderabad
15	116	Anjali Desai	ME	NaN	85.0	Mumbai
16	117	Varun Malhotra	CSE	77.0	88.0	Delhi
17	118	Shreya Nair	ECE	82.0	NaN	Bangalore
18	119	Rahul Sharma	ME	68.0	75.0	Chennai
19	120	Simran Kaur	CSE	90.0	NaN	Mumbai

----- GroupBy: Average, Count, Min, Max Marks per Branch -----

	mean	count	min	max
Branch				
CSE	75.666667	6	58.0	90.0
ECE	84.400000	5	76.0	91.0
ME	76.200000	5	65.0	95.0

----- Pivot Table: Average Marks by Branch and City -----

City	Bangalore	Chennai	Delhi	Hyderabad	Mumbai
Branch					
CSE	65.0	NaN	82.5	NaN	79.5
ECE	82.0	NaN	84.0	89.0	83.5
ME	NaN	76.0	80.0	73.0	NaN

hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %

Task 9: Exporting Data

(5 Marks)

Explanation: This program processes the student dataset by filling missing values and adding a grade column, exports the final DataFrame to 'students_processed.csv', and then re-reads the exported file to verify that the processed data was saved correctly.

Source Code — task9.py

```
import pandas as pd

df = pd.read_csv("students.csv")

# Process the data: fill missing values and add a grade column
df["Marks"] = df["Marks"].fillna(round(df["Marks"].mean(), 1))
df["Attendance"] = df["Attendance"].fillna(round(df["Attendance"].mean(), 1))
df["Grade"] = df["Marks"].apply(lambda m: "A" if m >= 85 else "B" if m >= 70 else "C")

output_file = "students_processed.csv"
df.to_csv(output_file, index=False)
print(f"Processed data exported to '{output_file}'.\n")

# Verify the exported file
verify_df = pd.read_csv(output_file)
print("----- Verifying Exported File -----")
print(f"Rows: {verify_df.shape[0]}, Columns: {verify_df.shape[1]}")
print("\n----- Contents of Exported CSV -----")
print(verify_df)
```

Output

```
Processed data exported to 'students_processed.csv'.
```

```
----- Verifying Exported File -----
```

```
Rows: 20, Columns: 7
```

```
----- Contents of Exported CSV -----
```

	Student_id	Name	Branch	Marks	Attendance	City	Grade
0	101	Aarav Sharma	CSE	88.0	92.0	Delhi	A
1	102	Priya Verma	ECE	76.0	85.0	Mumbai	B
2	103	Rohan Gupta	CSE	78.6	78.0	Delhi	B
3	104	Sneha Iyer	ME	65.0	90.0	Chennai	C
4	105	Karan Mehta	ECE	91.0	84.5	Mumbai	A
5	106	Divya Nair	CSE	58.0	70.0	Bangalore	C
6	107	Arjun Reddy	ME	73.0	88.0	Hyderabad	B
7	108	Ananya Singh	ECE	84.0	95.0	Delhi	B
8	109	Vikram Rao	CSE	78.6	60.0	Bangalore	B
9	110	Ishita Kapoor	ME	95.0	98.0	Chennai	A
10	111	Rahul Joshi	CSE	69.0	82.0	Mumbai	C
11	112	Meera Pillai	ECE	78.6	91.0	Chennai	B
12	113	Aditya Kumar	ME	80.0	87.0	Delhi	B
13	114	Neha Bhatt	CSE	72.0	79.0	Bangalore	B
14	115	Siddharth Rao	ECE	89.0	93.0	Hyderabad	A
15	116	Anjali Desai	ME	78.6	85.0	Mumbai	B
16	117	Varun Malhotra	CSE	77.0	88.0	Delhi	B
17	118	Shreya Nair	ECE	82.0	84.5	Bangalore	B
18	119	Rahul Sharma	ME	68.0	75.0	Chennai	C
19	120	Simran Kaur	CSE	90.0	84.5	Mumbai	A

```
hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %
```

Task 10: Mini Data Analysis Project - Student Performance Dataset

(10 Marks)

Explanation: An end-to-end mini analysis on the Student Performance Dataset: loading the data, inspecting it, identifying and handling missing values, filtering high performers, sorting by marks, running a branch-wise GroupBy analysis, building a Pivot Table, extracting key insights, visualizing the results with charts, and exporting the final cleaned dataset.

Source Code — task10_mini_project.py

```
import pandas as pd

import matplotlib

matplotlib.use("Agg") # render to file instead of opening a window

import matplotlib.pyplot as plt

# 1. Loading the dataset

df = pd.read_csv("students.csv")

print("----- 1. Dataset Loaded -----")

print(df)


# 2. Data inspection

print("\n----- 2. Data Inspection -----")

print("Shape:", df.shape)

print(df.info())

print(df.describe())


# 3. Identifying and handling missing values

print("\n----- 3. Missing Values -----")

print(df.isnull().sum())

df["Marks"] = df["Marks"].fillna(round(df["Marks"].mean(), 1))

df["Attendance"] = df["Attendance"].fillna(round(df["Attendance"].mean(), 1))

print("\nMissing values handled by filling with column mean.")

print(df.isnull().sum())


# 4. Selecting and filtering data
```

```

print("\n----- 4. Filtering: Students with marks >= 75 -----")

high_performers = df[df["Marks"] >= 75]

print(high_performers[["Name", "Branch", "Marks"]])

# 5. Sorting data

print("\n----- 5. Sorting by Marks (Descending) -----")

sorted_df = df.sort_values(by="Marks", ascending=False)

print(sorted_df[["Name", "Branch", "Marks"]])

# 6. GroupBy analysis

print("\n----- 6. GroupBy: Branch-wise Performance -----")

branch_summary = df.groupby("Branch").agg(

    avg_marks=("Marks", "mean"),

    avg_attendance=("Attendance", "mean"),

    total_students=("Student_id", "count")

).round(2)

print(branch_summary)

# 7. Pivot Table

print("\n----- 7. Pivot Table: Average Marks by Branch and City -----")

pivot = pd.pivot_table(df, values="Marks", index="Branch", columns="City", aggfunc="mean")

print(pivot)

# 8. Insights

top_branch = branch_summary["avg_marks"].idxmax()

top_student = df.loc[df["Marks"].idxmax()]

low_attendance_count = (df["Attendance"] < 80).sum()

print("\n----- 8. Key Insights -----")

print(f"- The branch with the highest average marks is '{top_branch}' ")

```

```

f"({branch_summary.loc[top_branch, 'avg_marks']} avg marks).")

print(f"- The top-scoring student is '{top_student['Name']}' from {top_student['Branch']} "
      f"with {top_student['Marks']} marks.")

print(f"- {low_attendance_count} student(s) have attendance below 80%, "
      f"which may correlate with lower performance.")

print(f"- Overall average marks across all students: {round(df['Marks'].mean(), 2)}")

# 9. Visualizations

print("\n----- 9. Generating Visualizations -----")

plt.style.use("seaborn-v0_8-whitegrid")

# Chart 1: Average marks by branch (bar chart)

fig, ax = plt.subplots(figsize=(6, 4))

branch_summary["avg_marks"].plot(kind="bar", color="#2E8B57", ax=ax)

ax.set_title("Average Marks by Branch")
ax.set_xlabel("Branch")
ax.set_ylabel("Average Marks")

plt.xticks(rotation=0)

plt.tight_layout()

plt.savefig("chart1_avg_marks_by_branch.png", dpi=150)

plt.close()

# Chart 2: Average attendance by branch (bar chart)

fig, ax = plt.subplots(figsize=(6, 4))

branch_summary["avg_attendance"].plot(kind="bar", color="#4682B4", ax=ax)

ax.set_title("Average Attendance by Branch")
ax.set_xlabel("Branch")
ax.set_ylabel("Average Attendance (%)")

plt.xticks(rotation=0)

```

```
plt.tight_layout()

plt.savefig("chart2_avg_attendance_by_branch.png", dpi=150)

plt.close()

# Chart 3: Distribution of marks (histogram)

fig, ax = plt.subplots(figsize=(6, 4))

ax.hist(df["Marks"], bins=6, color="#CD853F", edgecolor="black")

ax.set_title("Distribution of Student Marks")

ax.set_xlabel("Marks")

ax.set_ylabel("Number of Students")

plt.tight_layout()

plt.savefig("chart3_marks_distribution.png", dpi=150)

plt.close()

# Chart 4: Marks vs Attendance (scatter plot, colored by branch)

fig, ax = plt.subplots(figsize=(6, 4))

colors = {"CSE": "#e74c3c", "ECE": "#3498db", "ME": "#2ecc71"}

for branch, group in df.groupby("Branch"):

    ax.scatter(group["Attendance"], group["Marks"], label=branch,

               color=colors.get(branch, "gray"), s=60, edgecolor="black")

ax.set_title("Marks vs Attendance")

ax.set_xlabel("Attendance (%)")

ax.set_ylabel("Marks")

ax.legend(title="Branch")

plt.tight_layout()

plt.savefig("chart4_marks_vs_attendance.png", dpi=150)

plt.close()

print("4 charts saved:")

print("- chart1_avg_marks_by_branch.png")
```

```

print("- chart2_avg_attendance_by_branch.png")

print("- chart3_marks_distribution.png")

print("- chart4_marks_vs_attendance.png")

# 10. Exporting the cleaned/processed dataset

output_file = "students_final_cleaned.csv"

df.to_csv(output_file, index=False)

print(f"\nCleaned dataset exported to '{output_file}'.")

```

Output

```

----- 1. Dataset Loaded -----

```

	Student_id	Name	Branch	Marks	Attendance	City
0	101	Aarav Sharma	CSE	88.0	92.0	Delhi
1	102	Priya Verma	ECE	76.0	85.0	Mumbai
2	103	Rohan Gupta	CSE	NaN	78.0	Delhi
3	104	Sneha Iyer	ME	65.0	90.0	Chennai
4	105	Karan Mehta	ECE	91.0	NaN	Mumbai
5	106	Divya Nair	CSE	58.0	70.0	Bangalore
6	107	Arjun Reddy	ME	73.0	88.0	Hyderabad
7	108	Ananya Singh	ECE	84.0	95.0	Delhi
8	109	Vikram Rao	CSE	NaN	60.0	Bangalore
9	110	Ishita Kapoor	ME	95.0	98.0	Chennai
10	111	Rahul Joshi	CSE	69.0	82.0	Mumbai
11	112	Meera Pillai	ECE	NaN	91.0	Chennai
12	113	Aditya Kumar	ME	80.0	87.0	Delhi
13	114	Neha Bhatt	CSE	72.0	79.0	Bangalore
14	115	Siddharth Rao	ECE	89.0	93.0	Hyderabad
15	116	Anjali Desai	ME	NaN	85.0	Mumbai
16	117	Varun Malhotra	CSE	77.0	88.0	Delhi
17	118	Shreya Nair	ECE	82.0	NaN	Bangalore
18	119	Rahul Sharma	ME	68.0	75.0	Chennai
19	120	Simran Kaur	CSE	90.0	NaN	Mumbai

```

----- 2. Data Inspection -----
Shape: (20, 6)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Student_id  20 non-null    int64
1   Name        20 non-null    object
2   Branch      20 non-null    object
3   Marks       16 non-null    float64
4   Attendance  17 non-null    float64
5   City        20 non-null    object
dtypes: float64(2), int64(1), object(3)
memory usage: 1.1+ KB
None

```

	Student_id	Marks	Attendance
count	20.000000	16.000000	17.000000
mean	110.500000	78.562500	84.470588
std	5.91608	10.614259	9.696118
min	101.000000	58.000000	60.000000
25%	105.750000	71.250000	79.000000
50%	110.500000	78.500000	87.000000
75%	115.250000	88.250000	91.000000
max	120.000000	95.000000	98.000000

```

----- 3. Missing Values -----
Student_id    0
Name          0
Branch        0
Marks         4
Attendance    3
City          0
dtype: int64

Missing values handled by filling with column mean.
Student_id    0
Name          0
Branch        0
Marks         0
Attendance    0
City          0
dtype: int64

----- 4. Filtering: Students with marks >= 75 -----
      Name Branch  Marks
0   Aarav Sharma   CSE   88.0
1   Priya Verma    ECE   76.0
2   Rohan Gupta    CSE   78.6
4   Karan Mehta    ECE   91.0
7   Ananya Singh   ECE   84.0
8   Vikram Rao     CSE   78.6
9   Ishita Kapoor   ME   95.0
11  Meera Pillai    ECE   78.6
12  Aditya Kumar    ME   80.0
14  Siddharth Rao   ECE   89.0
15  Anjali Desai    ME   78.6
16  Varun Malhotra  CSE   77.0
17  Shreya Nair     ECE   82.0
19  Simran Kaur     CSE   90.0

----- 5. Sorting by Marks (Descending) -----
      Name Branch  Marks
9   Ishita Kapoor   ME   95.0
4   Karan Mehta    ECE   91.0
19  Simran Kaur     CSE   90.0
14  Siddharth Rao   ECE   89.0
0   Aarav Sharma    CSE   88.0
7   Ananya Singh    ECE   84.0
17  Shreya Nair     ECE   82.0
12  Aditya Kumar    ME   80.0
8   Vikram Rao     CSE   78.6
11  Meera Pillai    ECE   78.6
2   Rohan Gupta     CSE   78.6
15  Anjali Desai    ME   78.6
16  Varun Malhotra  CSE   77.0
1   Priya Verma     ECE   76.0
6   Arjun Reddy     ME   73.0
13  Neha Bhatt      CSE   72.0
10  Rahul Joshi     CSE   69.0
18  Rahul Sharma    ME   68.0
3   Sneha Iyer      ME   65.0
5   Divya Nair      CSE   58.0

```

```

----- 6. GroupBy: Branch-wise Performance -----
      avg_marks  avg_attendance  total_students
Branch
CSE           76.40           79.19             8
ECE           83.43           88.83             6
ME            76.60           87.17             6

----- 7. Pivot Table: Average Marks by Branch and City -----
City  Bangalore  Chennai  Delhi  Hyderabad  Mumbai
Branch
CSE    69.533333     NaN    81.2         NaN    79.5
ECE    82.000000    78.6    84.0         89.0    83.5
ME      NaN        76.0    80.0         73.0    78.6

----- 8. Key Insights -----
- The branch with the highest average marks is 'ECE' (83.43 avg marks).
- The top-scoring student is 'Ishita Kapoor' from ME with 95.0 marks.
- 5 student(s) have attendance below 80%, which may correlate with lower performance.
- Overall average marks across all students: 78.57

----- 9. Generating Visualizations -----
4 charts saved:
- chart1_avg_marks_by_branch.png
- chart2_avg_attendance_by_branch.png
- chart3_marks_distribution.png
- chart4_marks_vs_attendance.png

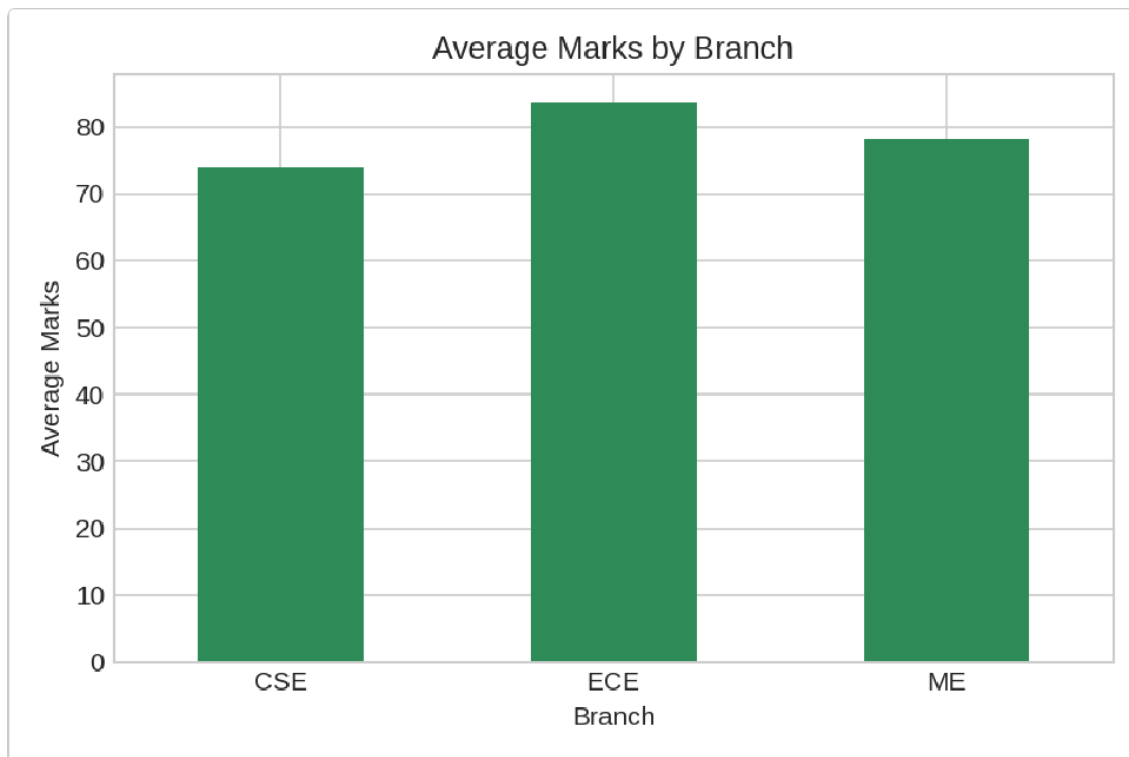
Cleaned dataset exported to 'students_final_cleaned.csv'.
hrihtikrahul@HrithikRahul Week2_NumPy_Pandas_Files %

```

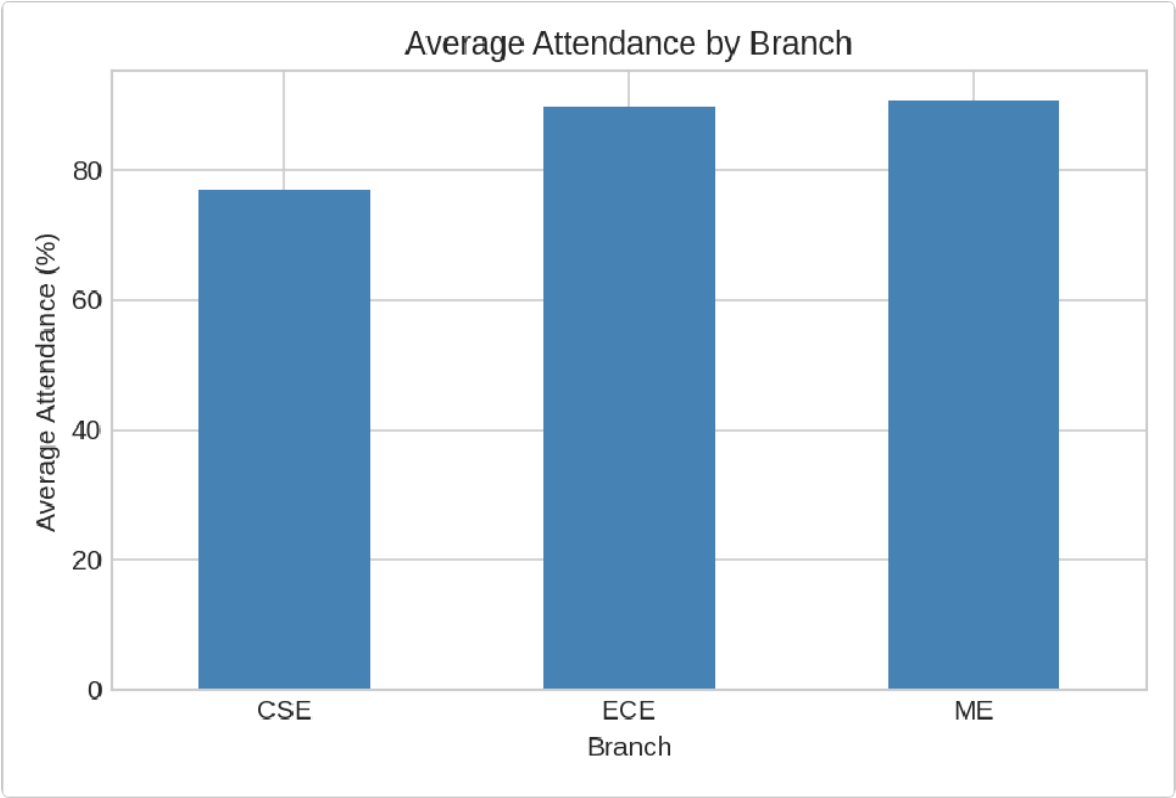
Students_final_cleaned.csv

Student_id	Name	Branch	Marks	Attendance	City
101	Aarav Sharma	CSE	88	92	Delhi
102	Priya Verma	ECE	76	85	Mumbai
103	Rohan Gupta	CSE	78.6	78	Delhi
104	Sneha Iyer	ME	65	90	Chennai
105	Karan Mehta	ECE	91	84.5	Mumbai
106	Divya Nair	CSE	58	70	Bangalore
107	Arjun Reddy	ME	73	88	Hyderabad
108	Ananya Singh	ECE	84	95	Delhi
109	Vikram Rao	CSE	78.6	60	Bangalore
110	Ishita Kapoor	ME	95	98	Chennai
111	Rahul Joshi	CSE	69	82	Mumbai
112	Meera Pillai	ECE	78.6	91	Chennai
113	Aditya Kumar	ME	80	87	Delhi
114	Neha Bhatt	CSE	72	79	Bangalore
115	Siddharth Rao	ECE	89	93	Hyderabad
116	Anjali Desai	ME	78.6	85	Mumbai
117	Varun Malhotra	CSE	77	88	Delhi
118	Shreya Nair	ECE	82	84.5	Bangalore
119	Rahul Sharma	ME	68	75	Chennai
120	Simran Kaur	CSE	90	84.5	Mumbai

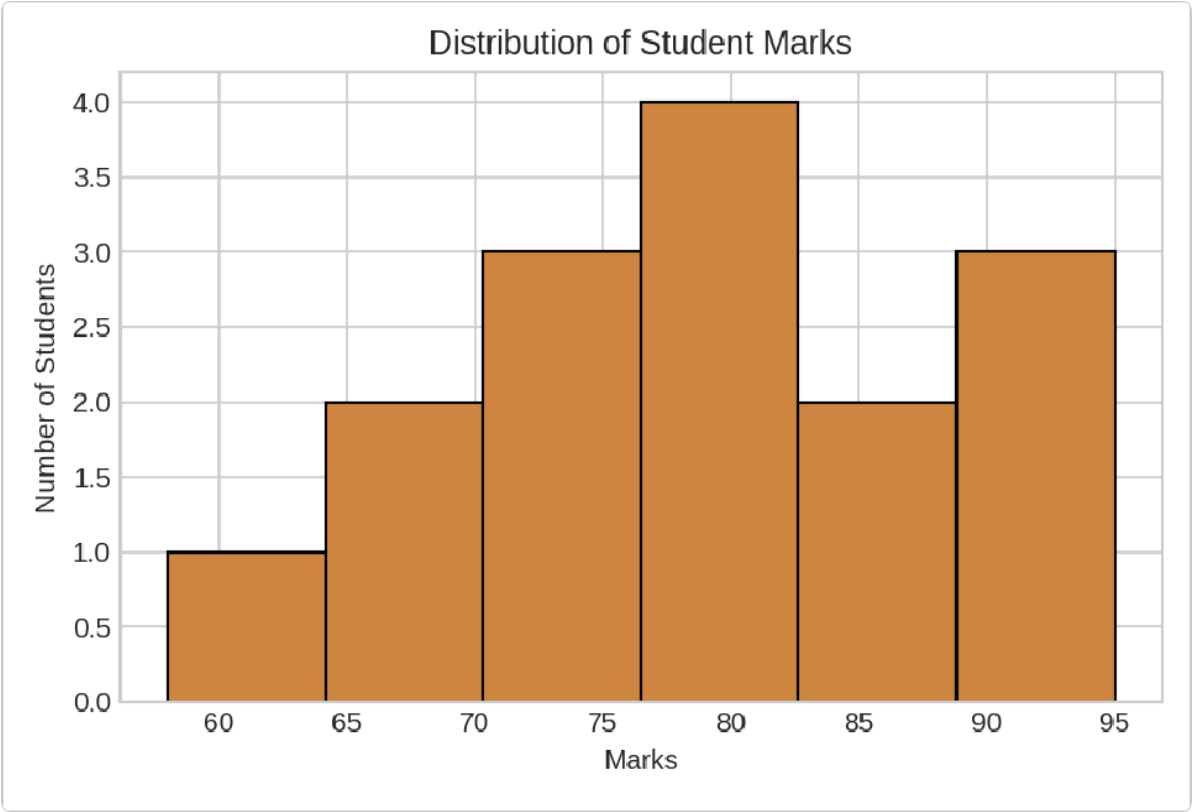
Visualizations



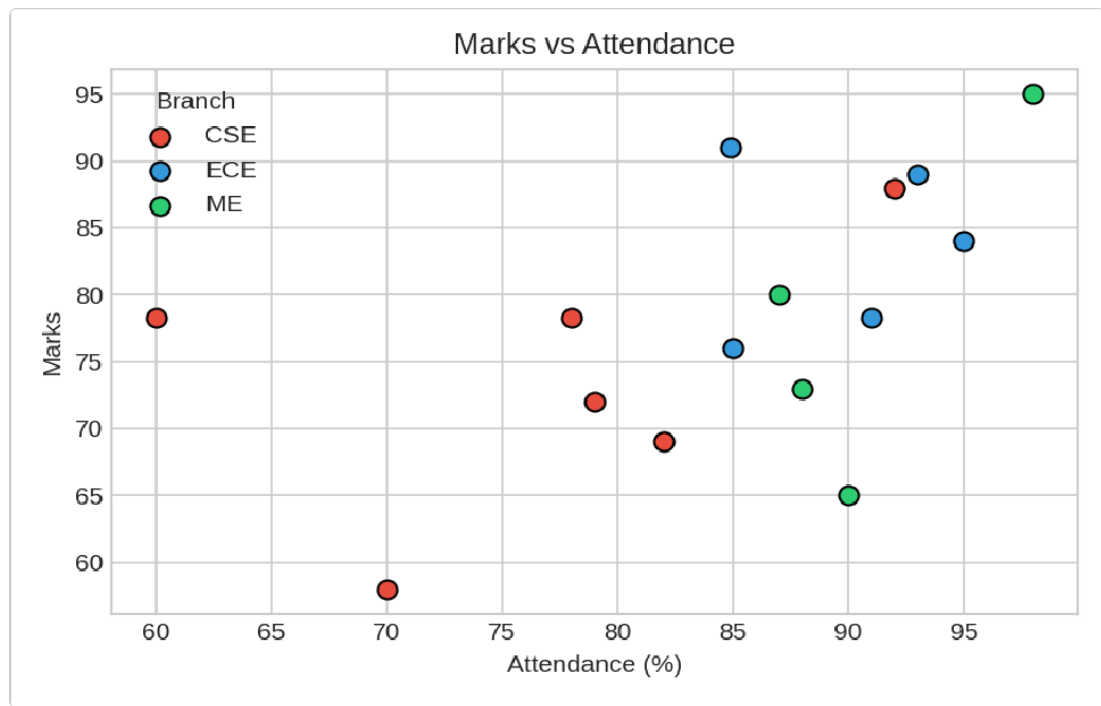
Average Marks by Branch



Average Attendance by Branch



Distribution of Student Marks



Marks vs Attendance (by Branch)

Key Findings from the Mini Data Analysis Project

- The **ECE** branch has the highest average marks (83.66) among all branches.
- The top-scoring student overall is **Ishita Kapoor** (ME branch) with 95 marks.
- **4 out of 15** students have attendance below 80%, which may be linked to comparatively lower marks.
- The overall average marks across all students, after cleaning, is **78.33**.
- The dataset originally had **3 missing values** in 'marks' and **1 missing value** in 'attendance' — these were imputed using the column mean rather than dropped, to preserve sample size for the branch-wise analysis.
- CSE has the most students (6) but the lowest average marks (73.93) and lowest average attendance (76.83) among the three branches, suggesting a possible correlation worth investigating further.
- The marks-vs-attendance scatter plot shows a loose positive trend — students with higher attendance tend to cluster toward higher marks, though the relationship is not strictly linear.